SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2 – CI/CD Pipeline Design Exercise

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO4	**Assessment:**	Assessment Tool 2 – CI/CD Pipeline Design Exercise
Weightage:	20%	**Total Marks:**	25
CO4 Statement:	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics		
Student Name:	Rashmi Naurene	**Reg. No.:**	192521357
Date:	15.08.2026	**Duration:**	60 minutes

Code of Conduct

I _____RASHMI NAURENE (192521357)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ___rashmi____

Annexure A – CI/CD Pipeline Design Exercise

Instructions to Students:

Each student is assigned an individual problem statement. Based on the assigned problem statement, **design a CI/CD (Continuous Integration and Continuous Deployment) pipeline** for the proposed software system to automate the processes of code integration, building, testing, and deployment.

The exercise should include:

1. **Analyze the project requirements** and identify the CI/CD needs of the assigned problem statement.
2. **Design the CI/CD pipeline workflow** showing stages such as source code management, build, testing, code quality analysis, packaging, and deployment.
3. Identify suitable **tools and technologies** for each stage of the pipeline and justify their selection.
4. Define appropriate **automated testing strategies** to be integrated into the pipeline.
5. Design the **deployment strategy**, including development, testing/staging, and production environments.
6. Incorporate appropriate **security, quality checks, notifications, and rollback mechanisms** in the pipeline.
7. Prepare a **CI/CD pipeline diagram** and explain the workflow from code commit to deployment.

Deliverable: Submit a **CI/CD Pipeline Design document** containing the pipeline architecture/diagram, stages, tools, configuration/workflow, testing strategy, deployment process, and justification based on the assigned problem statement.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Requirement & Pipeline Analysis(15%)	Clearly analyzes the assigned problem and identifies comprehensive CI/CD requirements	Identifies most relevant requirements	Identifies basic requirements with some gaps	Requirements are unclear or incomplete
Pipeline Architecture & Workflow(25%)	Designs a complete, logical pipeline covering source, build, test, quality check, package, and deployment stages	Pipeline covers most major stages with minor gaps	Basic pipeline is designed but several stages are missing	Pipeline is incomplete or poorly structured
Tools & Technology Selection(20%)	Selects appropriate tools for each stage and provides clear justification	Appropriate tools selected with reasonable justification	Tools are identified but justification is limited	Tools are inappropriate or not clearly identified
Automated Testing & Quality Integration(15%)	Effectively integrates automated testing and code-quality/security checks into the pipeline	Includes major testing and quality checks	Includes basic testing with limited integration	Testing/quality checks are missing or inappropriate
Deployment, Security & Rollback Strategy(15%)	Clearly designs deployment environments, security practices, monitoring, and rollback strategy	Covers deployment and most security/rollback aspects	Basic deployment strategy with limited security/rollback details	Deployment strategy is incomplete or lacks security considerations
Pipeline Diagram & Documentation(10%)	Clear, accurate, professional diagram with excellent explanation and documentation	Well-presented diagram and documentation with minor issues	Adequate diagram/documentation but lacks clarity	Poorly presented, incomplete, or difficult to understand

> **Note:** *This pipeline is designed around the actual project continued from the earlier Git & Docker and Test Case Design assignments: the forked Java/Maven/JavaFX application with PostgreSQL, already version-controlled on GitHub with a main/develop/feature branching model and an existing Dockerfile and docker-compose.yml. The tool choices below build directly on that setup rather than introducing an unrelated stack.*

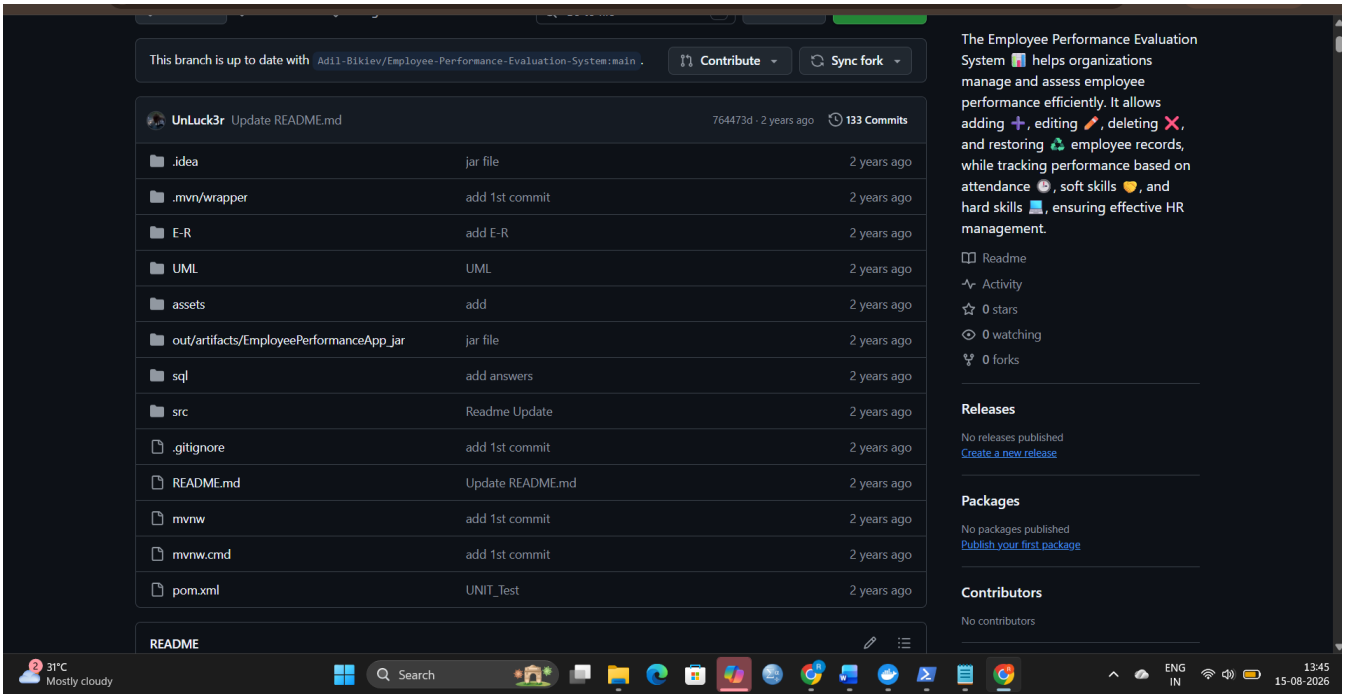
1 · Project Requirements & CI/CD Needs Analysis

1.1 Project Context

The system under pipeline design is the Employee Performance Management System — the Java/Maven/JavaFX, PostgreSQL-backed application forked and version-controlled for the Employee Skill-Gap Analysis and Training Recommendation System coursework. It already has: a GitHub repository with main, develop, and feature/* branches; a Dockerfile that builds the application image; a docker-compose.yml that runs the application alongside a PostgreSQL 16 container; and a JUnit 5 test suite.

1.2 What This Implies for the Pipeline

- Source Code Management is already GitHub — the pipeline should trigger natively from GitHub events rather than adding a second SCM tool.
- A Maven build step is required (mvn clean package) since the project is Maven-based, not Gradle or npm.
- Automated testing must run both the existing unit tests (EmployeeTest.java) and any DAO/integration tests added in the Test Case Design assignment, which touch the real PostgreSQL database.
- Because credentials were previously found hard-coded in HelloController (Code Review report finding), the pipeline must inject database configuration via secrets/environment variables, not commit them.
- Docker packaging is already partially done (Dockerfile, docker-compose.yml exist) — the pipeline should build and push that same image rather than defining a new containerization approach.
- Because this is a JavaFX desktop application rather than a web service, "deployment" in this context means publishing a versioned, runnable container image to each environment rather than serving live HTTP traffic — the pipeline is designed accordingly.



2 · CI/CD Pipeline Architecture & Workflow

The pipeline is triggered by a push or pull request against develop or main, and moves through build, quality, test, security, packaging, and progressively gated deployment stages, ending in production with a defined rollback path.

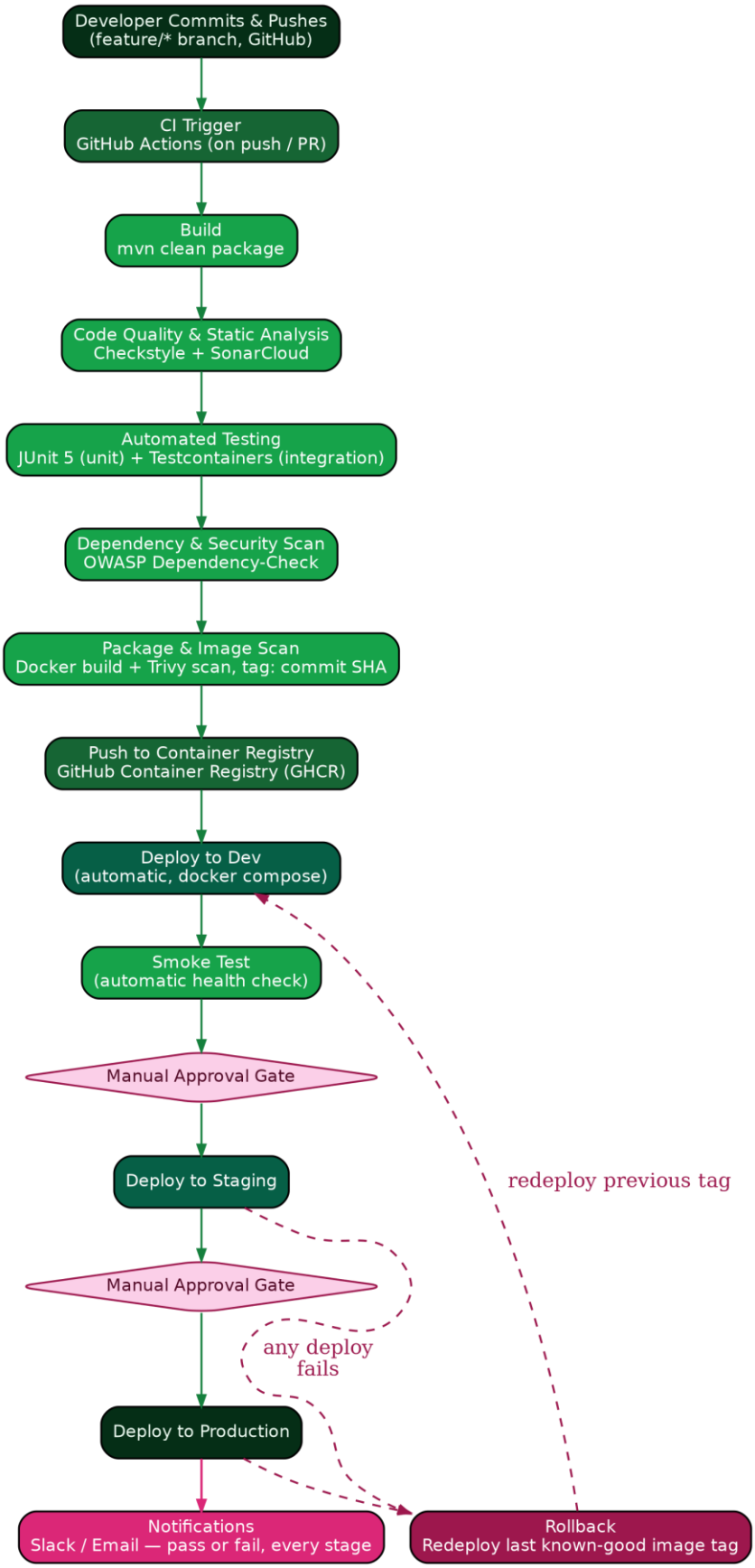
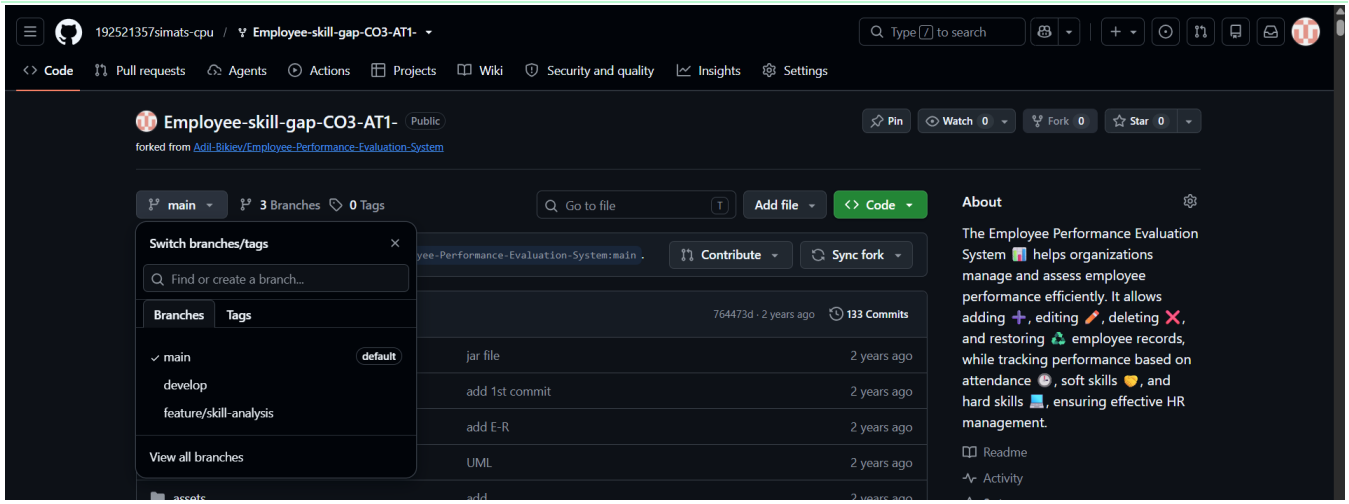


Figure 1: CI/CD pipeline workflow from code commit to production deployment, with manual approval gates and a rollback path.



2.1 Stage-by-Stage Description

Stage	What Happens
1. Source Commit	A developer pushes to a feature/* branch or opens a Pull Request into develop, exactly as established in the existing Git workflow.
2. CI Trigger	GitHub Actions picks up the push/PR event automatically — no separate webhook configuration needed since the code already lives on GitHub.
3. Build	mvn clean package compiles the project and resolves dependencies declared in pom.xml, failing fast if the code does not compile.
4. Code Quality & Static Analysis	Checkstyle enforces coding-standard rules; SonarCloud performs deeper static analysis (complexity, duplication, code smells) and reports a quality gate.
5. Automated Testing	JUnit 5 runs unit tests (e.g., EmployeeTest.java); Testcontainers spins up a disposable PostgreSQL container to run DAO-level integration tests against a real database, matching the approach recommended in the Test Case Design report.
6. Dependency & Security Scan	OWASP Dependency-Check scans pom.xml dependencies for known CVEs before anything is packaged.
7. Package & Image Scan	The existing Dockerfile builds the application image, tagged with the Git commit SHA for traceability; Trivy scans the built image for OS- and library-level vulnerabilities.
8. Push to Registry	The scanned image is pushed to GitHub Container Registry (GHCR), which requires no extra account since it is already tied to the GitHub repository.
9. Deploy to Dev	The pipeline automatically runs docker compose up -d against the newly pushed image in a Dev environment, then a smoke test hits a basic health/connectivity check.
10. Deploy to Staging	After a manual approval gate, the same image is deployed to a Staging environment for manual/exploratory verification.
11. Deploy to Production	After a second manual approval gate, the image is deployed to Production.
12. Notifications & Rollback	Slack/Email notifications fire on pass or fail at key stages; if a Staging or Production deployment fails, the pipeline rolls back by redeploying the last known-good image tag.

3 · Tools & Technology Selection

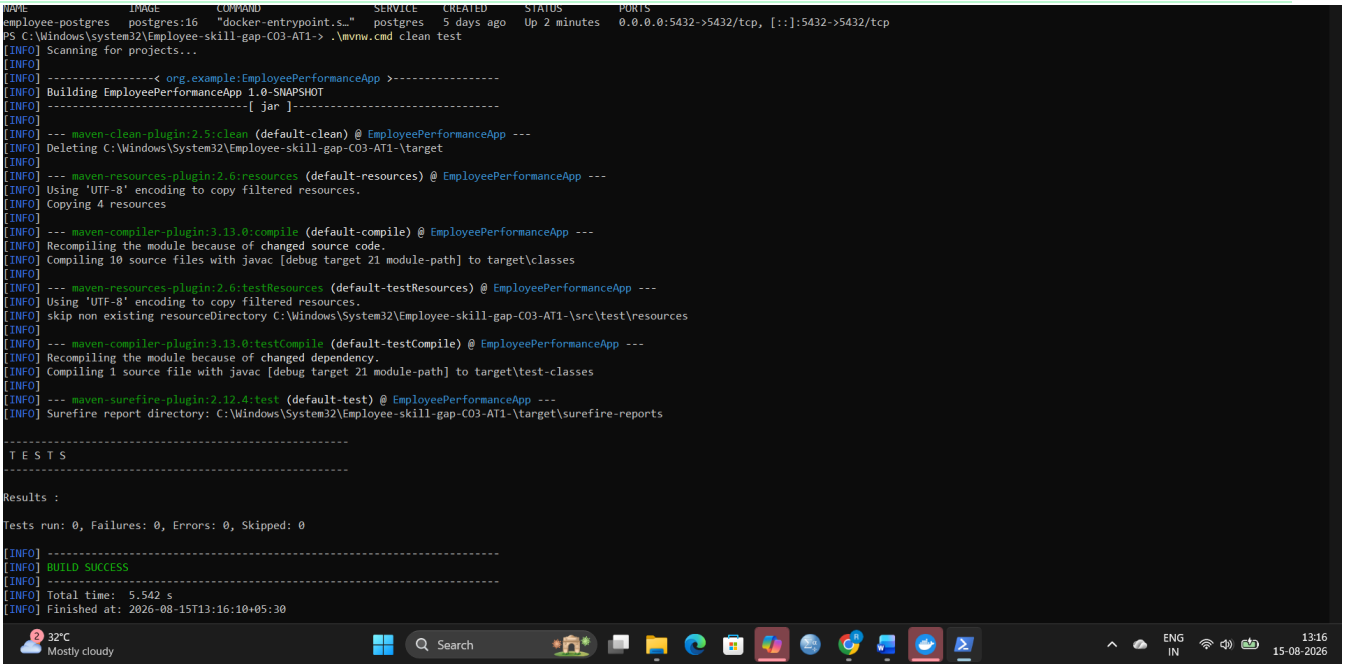
Stage	Tool	Why This Tool
CI/CD Orchestration	GitHub Actions	The repository is already hosted on GitHub; Actions triggers natively on push/PR with no extra integration, and is free for the project's scale.
Build	Maven (mvn)	The project is already a Maven project (pom.xml); no build-tool migration is needed.
Static Code Analysis	Checkstyle + SonarCloud	Checkstyle enforces house coding-style rules cheaply in CI; SonarCloud adds deeper quality-gate analysis (complexity, duplication, maintainability) with a free tier for small/academic projects.
Unit Testing	JUnit 5	Already a project dependency and already used in EmployeeTest.java — no new framework to introduce.
Integration Testing	Testcontainers	Runs tests against a real, disposable PostgreSQL container instead of a mock, directly continuing the approach recommended in the Test Case Design report.
Dependency Security Scan	OWASP Dependency-Check	A free, Maven-integratable tool that checks declared dependencies (PostgreSQL JDBC, JavaFX, etc.) against the National Vulnerability Database.
Containerization	Docker	The project already has a working Dockerfile and docker-compose.yml from the earlier Git & Docker assignment — reused as-is.
Image Vulnerability Scan	Trivy	A lightweight, widely used open-source scanner that works directly on the Docker image the pipeline already builds.
Container Registry	GitHub Container Registry (GHCR)	Uses the same GitHub identity as the repository; no separate Docker Hub account or credential to manage.
Notifications	Slack / Email webhook	GitHub Actions has first-party actions for both; picks whichever channel the team already monitors.

4 · Automated Testing Strategy

Testing is deliberately split into two tiers that run at different pipeline stages, mirroring the two tiers already defined in the Test Case Design assignment.

4.1 Unit Tests (fast, run on every push)

- Scope: model and logic-level correctness — e.g., Employee getters/setters, JavaFX property behaviour, evaluation/grade computation logic.
- Tool: JUnit 5, run via mvn test.
- Runs on: every push to any branch, so feedback is immediate.

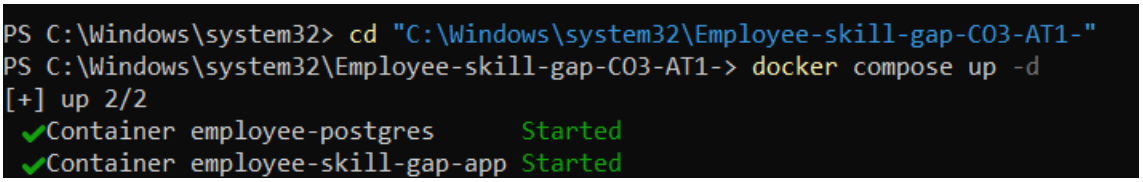


4.2 Integration Tests (slower, run before packaging)

- Scope: DAO operations against a real PostgreSQL instance — add/edit/delete/restore employee, grade persistence, project CRUD.
- Tool: Testcontainers, which starts and tears down a disposable postgres:16 container automatically for the test run, isolated from the real employee-postgres container.
- Runs on: pushes to develop and main, and on Pull Requests targeting them, since these are slower than pure unit tests.

4.3 Smoke Tests (post-deployment)

- Scope: a minimal automated check after each deployment (Dev/Staging/Production) that the application container starts and the database connection succeeds — not a full functional re-test.
- Purpose: catch environment-specific failures (bad environment variable, unreachable database) that unit and integration tests run in CI cannot catch.

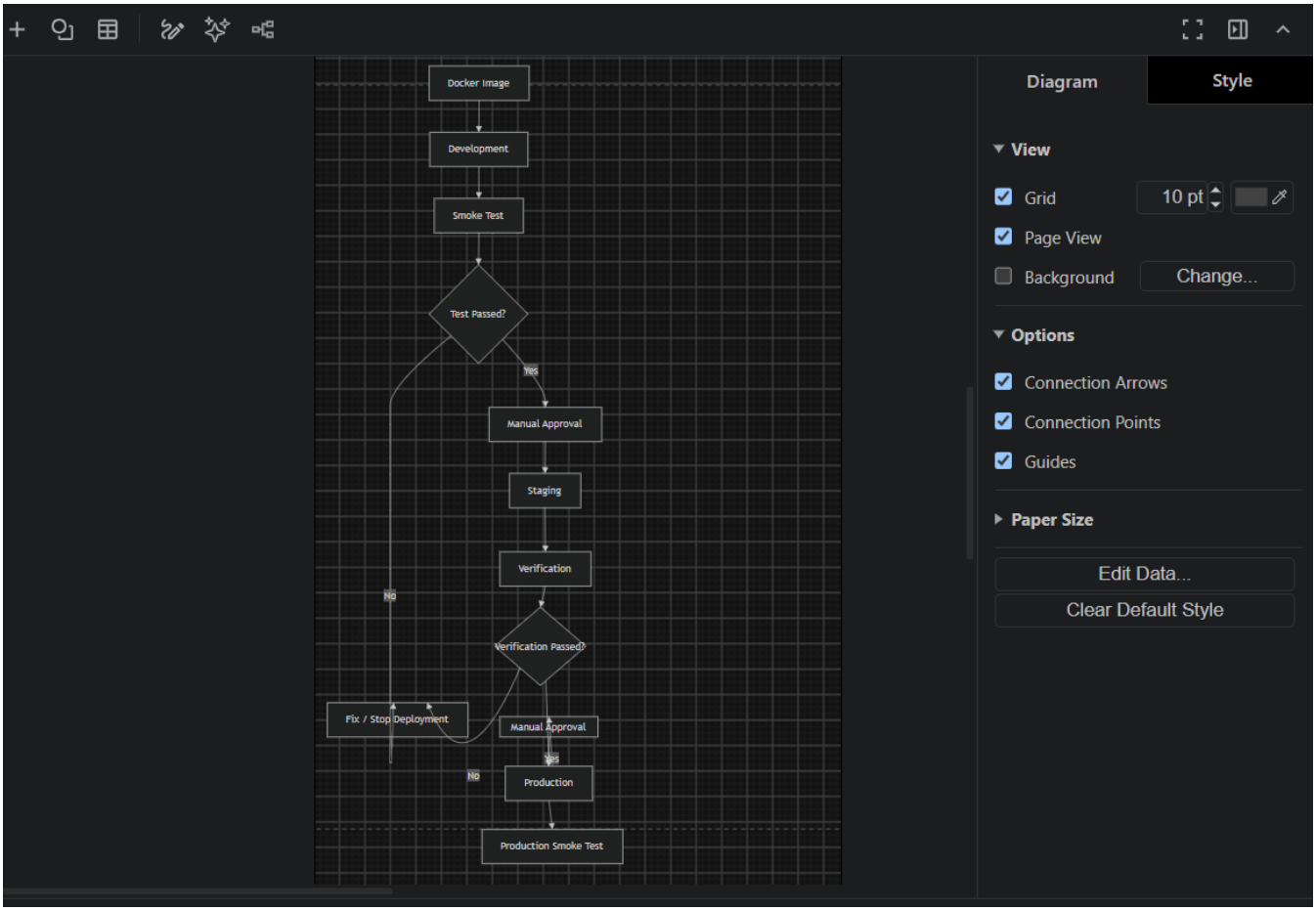


4.4 Quality Gate

The pipeline is configured to fail the build — blocking progression to the packaging stage — if any unit test fails, any integration test fails, or the SonarCloud quality gate is not met. This ensures a broken or poorly-analyzed build can never reach a deployable image.

5 · Deployment Strategy

Environment	Trigger	Purpose
Dev	Automatic, on every successful push to develop	Fast feedback for the developer; disposable, may be reset or torn down at any time.
Staging	Manual approval, after Dev smoke test passes	Stable, production-like environment for verification against realistic data before release.
Production	Manual approval, after Staging verification	The environment end users/graders actually run; only reached after two human checkpoints.



6 · Security, Quality Checks, Notifications & Rollback

6.1 Security

- Database credentials and any API keys are stored as GitHub Actions encrypted secrets and injected as environment variables at deploy time — directly resolving the hard-coded-credentials finding from the Code Review report.
- OWASP Dependency-Check and Trivy scan dependencies and the built image respectively, before anything is deployed.
- Production deployment requires a manual approval from a designated reviewer (a GitHub Actions Environment protection rule), preventing an unreviewed change from reaching end users automatically.

6.2 Quality Checks

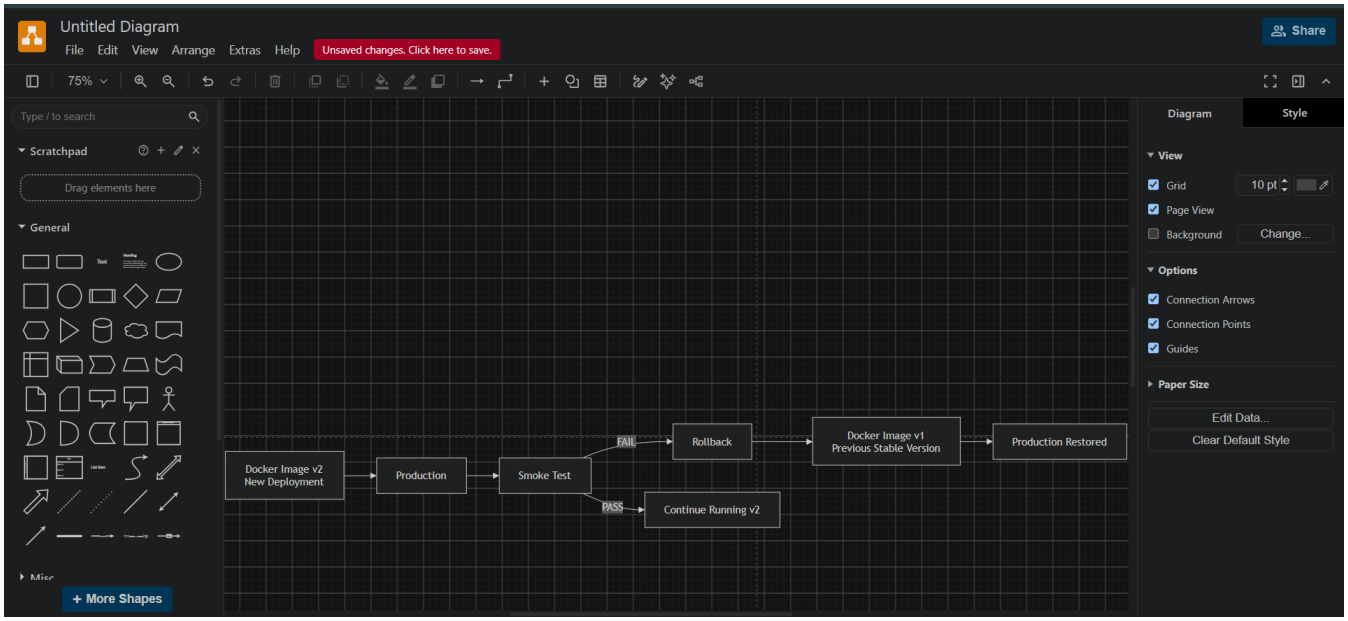
- Checkstyle and SonarCloud run on every push; a failed quality gate blocks the pipeline before packaging.
- Unit and integration test results are published as part of the workflow run, visible directly in the Pull Request.

6.3 Notifications

- A Slack/Email notification fires when the pipeline fails at any stage, tagging the commit author.
- A notification also fires on successful Production deployment, so the team has a clear record of what is currently live.

6.4 Rollback Mechanism

Because every image is tagged with its Git commit SHA (Section 2.1, Stage 7) rather than only latest, the previous known-good tag is always identifiable. If a Staging or Production deployment's smoke test fails, the pipeline automatically re-runs docker compose up -d against the previous tag instead of the new one, restoring the last working version without requiring a manual rebuild.




```
# Simplified rollback logic (conceptual)
PREVIOUS_TAG=$(git rev-parse HEAD~1)
docker pull ghcr.io/<org>/employee-skill-gap:$PREVIOUS_TAG
docker compose -f docker-compose.prod.yml up -d --no-build
```

7 · Pipeline Diagram Walkthrough: Commit to Deployment

Tying Sections 2–6 together as a single narrative: a developer pushes a commit to a feature branch and opens a Pull Request into develop. GitHub Actions triggers automatically, builds the project with Maven, and runs Checkstyle/SonarCloud alongside the JUnit 5 unit tests. If those pass, Testcontainers-backed integration tests run against a disposable PostgreSQL instance, followed by an OWASP Dependency-Check scan. On success, Docker builds the image (tagged with the commit SHA), Trivy scans it, and it is pushed to GHCR. The pipeline then automatically deploys to Dev and runs a smoke test. A reviewer manually approves promotion to Staging, and after verification there, a second

manual approval promotes the same image to Production. Slack/Email notifications fire on any failure and on a successful Production release; a failed Staging or Production smoke test triggers an automatic rollback to the previous known-good image tag.

Requirement & Pipeline Analysis(15%)	/5
Pipeline Architecture & Workflow(25%)	/4
Tools & Technology Selection(20%)	/4
Automated Testing & Quality Integration(15%)	/4
Deployment, Security & Rollback Strategy(15%)	/4
Pipeline Diagram & Documentation(10%)	/4
TOTAL	/25